

Self-Supervised Learning & CLIP: Teaching Machines to See Without Labels

KAUST–Mawhiba IOAI Training

Week 4 · Day 23



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 16, 2026

Part 1: Self-Supervised Learning

1. Bridge & Motivation
2. What is Self-Supervised Learning?
3. Pretext Tasks
4. Contrastive Learning
5. SimCLR
6. MoCo, BYOL, DINO
7. SSL Summary

Part 2: CLIP

8. CLIP Architecture
9. Zero-Shot Classification
10. Image Search with CLIP
11. What CLIP Enables
12. SSL in Practice
13. Day 23 Summary

What you already know:

- ▶ **Day 9–10:** CNNs learn features from *labeled* images (MNIST, ImageNet)
- ▶ **Day 10:** Transfer learning — borrow features from a pre-trained model
- ▶ **Day 14:** t-SNE/UMAP show how embeddings cluster in 2D
- ▶ **Day 17:** Transformers learn relationships between tokens
- ▶ **Day 19:** LoRA fine-tunes huge models cheaply

The common thread:

- ▶ Every model we trained needed *labeled data*
- ▶ Someone had to label each image, each sentence, each audio clip
- ▶ Labels are *expensive*: ImageNet took years and millions of dollars
- ▶ But there are *billions* of unlabeled images on the internet...

Today's question: Can we learn good features *without any labels at all*?

- ▶ **Supervised learning** has powered everything we have done so far:
 - MNIST: 60,000 images, each hand-labeled with a digit
 - ImageNet: 14 million images, each labeled by human annotators
 - Sentiment analysis: thousands of reviews labeled positive/negative
- ▶ **The bottleneck:** Getting labels is slow, expensive, and sometimes impossible
 - Medical images: you need a doctor to label each scan
 - Rare languages: you need native speakers
 - New domains: who labels satellite images of crop health?
- ▶ **Meanwhile:** Unlabeled data is *everywhere*
 - 3.2 billion images uploaded to social media every day
 - Millions of hours of video on YouTube
 - The entire internet is one giant unlabeled dataset

- ▶ **Self-supervised learning (SSL)** creates its own labels from the data
- ▶ The key idea: *hide part of the data, then predict it*
- ▶ You have already seen this without knowing it:
 - **BERT** (Day 18): mask a word, predict it from context — that is SSL!
 - **GPT** (Day 18): predict the next word — that is SSL!
 - **HuBERT** (Day 20): predict masked audio features — that is SSL!
- ▶ Today we bring this idea to *images*:
 - How do you “mask” an image? What do you “predict”?
 - How do you learn visual features without a single label?
 - And how does this lead to *CLIP* — a model that connects images and text?

- ▶ Let us put real numbers on the label problem:

Task	Cost per label	Time per label
ImageNet classification	\$0.01–0.05	2–5 seconds
Object detection (bboxes)	\$0.10–0.50	30–60 seconds
Semantic segmentation (pixels)	\$1.00–5.00	5–30 minutes
Medical image annotation	\$10–100	15–60 minutes
Autonomous driving (3D)	\$5–50	10–30 minutes

- ▶ **ImageNet** (14M images): cost $\sim$ \$500K and 2+ years
- ▶ **COCO segmentation** (330K instances): cost $\sim$ \$1.5M
- ▶ To label the 400M images CLIP uses? That would cost *\$4–200 million!*
- ▶ **SSL's promise:** Learn from all that data for *free*

Part 1: Self-Supervised Learning

1. What is SSL? The big picture
2. Pretext tasks: rotation, jigsaw, inpainting
3. Contrastive learning: the modern approach
4. SimCLR: a simple, powerful method
5. MoCo, BYOL, DINO: pushing further

Part 2: CLIP

6. From vision-only to vision + language
7. CLIP architecture: dual encoders
8. Zero-shot classification with CLIP
9. Image search with natural language
10. What CLIP enables: the bigger picture

Running Example

We will follow a single cat photo through every method today. Same image, different SSL approaches — watch how each one learns something different about it.

Supervised

Data + Labels

- ▶ “This is a cat”
- ▶ “This is a dog”
- ▶ Need humans to label
- ▶ Expensive, limited scale

Days 3–20

Unsupervised

Data only, no labels

- ▶ K-Means, PCA
- ▶ Find structure in data
- ▶ No prediction target
- ▶ Hard to evaluate

Days 14, 16

Self-Supervised

Data creates its own labels

- ▶ Hide part of data
- ▶ Predict the hidden part
- ▶ Free “labels” from data
- ▶ Scales to billions

Today!

Self-supervised = *supervised learning where the labels come from the data itself*

1. **Design a “pretext task”** — a puzzle the model must solve
 - The puzzle should be *hard enough* that the model must learn useful features
 - But *solvable* from the data alone (no human labels needed)
2. **Pre-train** the model on millions of unlabeled images
 - The model learns general visual features (edges, textures, shapes, objects)
 - Like ImageNet pre-training, but without the 14M labels
3. **Transfer** the learned features to your actual task
 - Freeze the encoder, add a small classifier head
 - Fine-tune on your (small) labeled dataset — same idea as Day 10!

Key Insight

The pretext task is *not* the goal. We care about the *features the model learns* while solving it.

- ▶ **Supervised:** Human provides the label
 - Input: image of a cat → Label: “cat” (human wrote this)
- ▶ **Self-supervised:** The data provides its own label
 - Image with a piece removed → Label: the missing piece
 - Rotated image → Label: the rotation angle (we chose it)
 - Two crops of same photo → Label: “these are the same”
- ▶ **The trick:** Transform or corrupt the data, ask the model to undo it. Free labels because *we created the puzzle and already know the answer*

Analogy

Like a fill-in-the-blank test — the teacher covers words in a sentence. The answers were right there!

- ▶ **BERT** (Day 18) is self-supervised:
 - Mask 15% of tokens, predict them
 - “The cat sat on the [MASK]” → Label: “mat” (from the sentence itself!)
- ▶ **GPT** (Day 18) is self-supervised:
 - Predict the next token
 - “The cat sat on the” → Label: “mat” (the next word!)
- ▶ **HuBERT** (Day 20) is self-supervised:
 - Mask audio frames, predict cluster IDs
 - Understands speech without transcriptions

Today: We do the same thing for *images*. But images are not sequences — so the pretext tasks look different.

Labeled data is scarce:

- ▶ ImageNet: 14M images, 21K classes
- ▶ Cost: \$25K+ and years of work
- ▶ Medical imaging: \$200/hour for a radiologist
- ▶ Most domains have <1000 labeled examples

Unlabeled data is abundant:

- ▶ Instagram: 100M+ photos uploaded daily
- ▶ YouTube: 500 hours of video per minute
- ▶ Web crawl: billions of images
- ▶ Cost: essentially free

- ▶ **The gap:** $1000\times$ more unlabeled data than labeled data
- ▶ **SSL bridges this gap:** Pre-train on unlabeled data, fine-tune on your small labeled set
- ▶ **Result:** Match or beat supervised models — with $10\times$ fewer labels

- ▶ A **pretext task** is a self-supervised puzzle we design for the model
- ▶ The model gets *no labels* — it has to figure out the answer from the image itself
- ▶ Three main families of pretext tasks:

Reconstructive

“Fix the broken image”

- ▶ Remove a piece
- ▶ Add noise
- ▶ Remove color
- ▶ Predict what is missing

Predictive

“Solve the visual puzzle”

- ▶ Predict rotation angle
- ▶ Solve a jigsaw
- ▶ Predict patch positions
- ▶ Predict the future frame

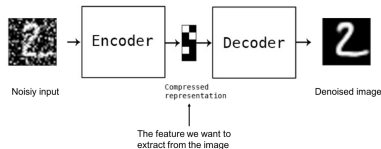
Contrastive

“Same or different?”

- ▶ Two crops of same image → similar
- ▶ Crops of different images → different
- ▶ Learn to tell them apart

Let us look at each family with concrete examples.

- ▶ **Idea:** Corrupt the image, then train a network to fix it
- ▶ **How:**
 1. Take an image
 2. Add random noise (or mask random patches)
 3. Feed the corrupted image to an encoder-decoder
 4. Train it to output the clean original
- ▶ **Why it works:** To denoise, the network must understand what belongs in the image — edges, textures, objects
- ▶ **Connection:** This is exactly the *autoencoder* from Day 11, just used differently!



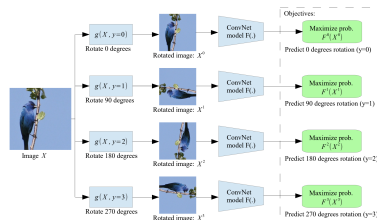
Corrupt → Encode → Decode →
Reconstruct

- ▶ **Idea:** Cut out a piece of the image; predict what was there
- ▶ **How:**
 1. Remove a rectangular patch from the center
 2. The model sees only the surrounding context
 3. It must “imagine” what the missing piece looks like
- ▶ **Why it works:** To fill in a gap, you need to understand:
 - Object shapes and parts
 - Scene layout
 - Color and texture patterns
- ▶ **Think of it as:** Visual “fill in the blank” — like BERT’s masked language modeling, but for images!



Pathak et al., 2016

- ▶ **Idea:** Rotate the image by 0, 90, 180, or 270 degrees; predict which rotation was applied
- ▶ **How:**
 1. Take any image
 2. Randomly rotate it (4 possible angles)
 3. Feed to a CNN
 4. Classify: 0°, 90°, 180°, or 270°?
- ▶ **Why it works:**
 - To know “this is upside down,” the model must know what “right-side up” looks like
 - It learns: sky is up, ground is down, faces have eyes above mouth
 - These are exactly the features useful for classification!



Gidaris et al., 2018

- ▶ **Idea:** Cut the image into patches, shuffle them, predict the original order
- ▶ **How:**
 1. Split image into a 3×3 grid (9 patches)
 2. Shuffle the patches randomly
 3. Feed each patch to a shared CNN
 4. Predict the correct arrangement
- ▶ **Why it works:**
 - The model learns spatial relationships
 - “This eye patch goes above this mouth patch”
 - “This sky patch goes above this tree patch”
 - It develops a sense of object geometry



Noroozi & Favaro, 2016

- ▶ **Try these yourself!** What features would a network need to learn?
 1. **Colorization:** Given a grayscale photo, predict the original colors
 - *Object identities: sky is blue, grass is green, skin tones*
 2. **Relative position:** Given two patches, predict if B is above/below/left/right of A
 - *Spatial structure: eyes above nose, wheels below car body*
 3. **Speed prediction:** Given a video clip, predict if it is sped up, slowed down, or normal
 - *How objects naturally move: people walk at a certain pace*

The cleverness is in *designing* the puzzle so the model *must* learn useful features to solve it.

- ▶ Rotation, jigsaw, and inpainting work, but they have limitations:
 - ▶ **Shortcut solutions:**
 - Rotation: cheat via JPEG artifacts. Jigsaw: match edges. Inpainting: copy nearby textures
 - ▶ **Task-specific features:**
 - Features for “predict rotation” are not always best for “classify this image”
 - ▶ **Limited transfer:**
 - These methods close only 50–70% of the gap with supervised ImageNet
- Enter contrastive learning** — the approach that changed everything.

- ▶ Instead of solving a specific puzzle, contrastive learning asks a simpler question:

“Are these two images *the same thing* or *different things*?”

Positive pair (same image):

- ▶ Take one photo of a cat
- ▶ Crop it two different ways
- ▶ These two crops should have *similar* embeddings

Negative pair (different images):

- ▶ A crop of a cat
- ▶ A crop of a car
- ▶ These should have *different* embeddings

Goal: Learn an embedding space where similar things are *close* and different things are *far apart*.

- ▶ **Key insight:** Each image is its own “class”
 - Two augmented views of the *same photo* = positive pair
 - Views of *any other photo* = negative pair
- ▶ **Data augmentations** create the positive pairs:
 - Random crop, color jitter, horizontal flip, Gaussian blur, grayscale
- ▶ **Why this works:**
 - A cat is still a cat whether you zoom in, flip it, or change the brightness
 - The model learns to ignore superficial changes (color, position)
 - It focuses on the *identity* of the object
 - These features transfer to real tasks like classification or detection

- ▶ Imagine a game: you see a photo and must find its match in a crowd
 - You have **1 correct match** (positive) and **99 wrong options** (negatives)
 - You win if you pick the correct one
- ▶ The **contrastive loss** (InfoNCE) trains the model to win this game:
 - Make the positive pair as **similar** as possible
 - Make all negative pairs as **different** as possible
 - Uses **cosine similarity** — same measure from Day 13!
- ▶ The harder the game (more negatives), the better the features

The formula (for reference)

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)}$$

sim = cosine similarity, τ = temperature

- ▶ Contrastive learning lives or dies by its **augmentations**
- ▶ The augmentations define what the model treats as “the same thing”

Augmentation	What it does	What the model learns
Random crop + resize	Zoom into different parts	Location does not matter
Horizontal flip	Mirror the image	Left-right does not matter
Color jitter	Change brightness/hue	Color is not identity
Gaussian blur	Make the image blurry	Shape > texture
Grayscale	Remove all color	Object shape, not color

Key Insight

By saying “this cropped, flipped, grayscale cat is the same as the original cat,” we force the model to learn *what makes a cat a cat* — not its color, position, or orientation.

- ▶ The temperature τ in InfoNCE controls how “picky” the model is:

Small τ (e.g., 0.07):

- ▶ Very sharp distinctions
- ▶ Small differences get amplified
- ▶ Can be unstable early on

Large τ (e.g., 1.0):

- ▶ Soft, smooth distinctions
- ▶ Treats many things as similar
- ▶ Easier to optimize

- ▶ **Analogy:** Temperature is like a game's difficulty setting
 - Low τ = hard mode: “tell me the *exact* match”
 - High τ = easy mode: “tell me anything *roughly* similar”
- ▶ **In practice:** $\tau = 0.07$ for SimCLR, $\tau = 0.01$ for CLIP

- ▶ **Before** contrastive training: embeddings are *random*
 - Cat images mixed with dog images, car images — no structure
- ▶ **After** contrastive training: embeddings form *meaningful clusters*
 - All cat images cluster together (different breeds, poses, lighting)
 - Similar categories (dogs vs. wolves) nearby but distinct
- ▶ **Remember Day 14:** t-SNE/UMAP shows contrastive learning creates beautiful clusters — without ever seeing a single label!
- ▶ **Why contrastive features transfer so well:**
 - Add a linear classifier on top — it draws boundaries between clusters
 - Few labeled examples are enough

► Compared to pretext tasks:

- No task-specific shortcuts (cannot cheat with JPEG artifacts)
- The model directly learns to tell things apart
- Features are general-purpose, useful for many tasks

► The augmentation is the secret sauce:

- “These two crops are the same cat” → model must figure out *what makes them the same*
- That means learning object identity, shape, texture

► Results: By 2020, contrastive learning matched or beat supervised ImageNet pre-training — all with *zero* labeled images!

SimCLR: Simple Contrastive Learning of Visual Representations

- ▶ **SimCLR** (Chen et al., 2020) is the clearest, most elegant contrastive method
- ▶ **Ingredients:**
 1. Strong data augmentations
 2. A CNN encoder (like ResNet-50)
 3. A small MLP “projection head”
 4. The InfoNCE contrastive loss
 5. Very large batch sizes
- ▶ **That is it.** No memory banks, no momentum encoders, no special tricks
- ▶ **Result:** 76.5% top-1 on ImageNet with linear evaluation (matches supervised!)

A Simple Framework for Contrastive Learning of Visual Representations

Ting Chen¹ Simon Kornblith¹ Mohammad Norouzi¹ Geoffrey Hinton¹

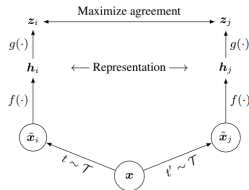
1. **Augment:** Take one image and create two random views
 - Random crop + resize, color jitter, Gaussian blur, horizontal flip
2. **Encode:** Pass both views through a shared encoder (e.g. ResNet-50)
 - Get representations h_i and h_j
3. **Project:** Pass through a small MLP projection head
 - Get z_i and z_j — loss is computed on z , but we keep h for real tasks
4. **Contrast:** In a batch of N images ($2N$ views), each positive pair is contrasted against $2(N-1)$ negatives using InfoNCE

Key: The encoder learns features that are the same for both views of the same image, but different for views of other images.

- ▶ Let us trace through one training step with a **batch of 4 images**:

Image	View A (crop + flip)	View B (crop + color change)
Cat photo	Cat-A	Cat-B
Dog photo	Dog-A	Dog-B
Car photo	Car-A	Car-B
Tree photo	Tree-A	Tree-B

- ▶ Now we have **8 views** total (2 per image)
- ▶ For Cat-A, the model must figure out:
 - *Match*: Cat-B (the other view of the same cat)
 - *Not a match*: Dog-A, Dog-B, Car-A, Car-B, Tree-A, Tree-B (6 distractors)
- ▶ The loss pushes Cat-A and Cat-B *together* and all others *apart*
- ▶ With real batch size 4096: each view has 1 match and **8190 distractors**!



- ▶ **Surprising finding:** The projection head is a 2-layer MLP that is *thrown away* after pre-training
- ▶ We use h (before projection) for real tasks, not z (after projection)
- ▶ **Why?** The projection head removes info useful for real tasks but not needed for the contrastive loss (like color, orientation)
- ▶ Think of it as: the projection head “absorbs” task-specific info, leaving h with the most general features

```

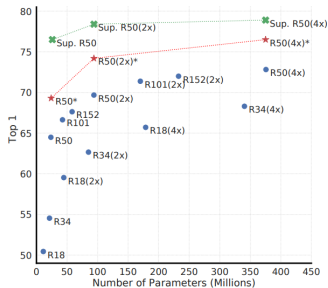
Input: batch size  $N$ , temperature  $\tau$ , structure of  $f, g, T$ 
for sampled mini-batch  $\{x_i\}_{i=1}^N$  do
  for all  $i \in \{1, \dots, N\}$  do
    draw two augmentation functions  $t \leftarrow T, t' \leftarrow T$ 
     $\tilde{x}_{i1} \leftarrow t(x_{i1})$  # first augmentation
     $\tilde{y}_{i1} \leftarrow g(\tilde{x}_{i1})$  # projection
     $h_{i1} \leftarrow f(\tilde{x}_{i1}, -)$  # representation
     $\tilde{x}_{i2} \leftarrow g(\tilde{h}_{i1}, -)$  # second augmentation
     $\tilde{x}_{i3} \leftarrow t'(\tilde{x}_{i2})$  # first augmentation
     $\tilde{y}_{i2} \leftarrow f(\tilde{x}_{i2}, -)$  # representation
     $\tilde{x}_{i4} \leftarrow g(\tilde{y}_{i2}, -)$  # projection
  end for
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do
     $s_{ij} \leftarrow z_i \cdot z_j / (v_i v_j)$  # pairwise similarity
  end for
  define  $\ell(f, j) \leftarrow \ell(f, j) - \log \frac{\exp(s_{ij})}{\sum_{k=1}^{2N} \exp(s_{ik})}$ 
   $\ell \leftarrow \frac{1}{2N} \sum_{i=1}^{2N} \ell(f, i)$ 
  update networks  $f$  and  $g$  to minimize  $\ell$ 
end for
return encoder network  $f$ 

```

- A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

```
simclr_augmentation = T.Compose([T.RandomResizedCrop(224, scale =  
(0.2, 1.0)), T.RandomHorizontalFlip(), T.RandomApply([T.ColorJitter(0.4, 0.4, 0.4, 0.1)], p =  
0.8), T.RandomGrayscale(p = 0.2), T.GaussianBlur(kernel_size =  
23), T.ToTensor(), T.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]),])  
Apply twice to get two different views view1 = simclr_augmentation(image) view2 =  
simclr_augmentation(image) view1 and view2 look different but are the same image
```

Same transforms as Day 9 data augmentation — but now they are the *core training signal*, not just a regularizer!



- ▶ SimCLR nearly matches supervised learning with *zero labels* during pre-training
- ▶ With only 1% of ImageNet labels for fine-tuning, SimCLR beats a supervised model trained on all labels from scratch
- ▶ This is the power of SSL: pre-train cheaply, fine-tune efficiently

- ▶ SimCLR showed contrastive learning works, but it needs huge batches
- ▶ Researchers asked: can we get the same quality with less compute?
- ▶ Three key innovations followed:

MoCo (2020)

“Use a memory bank”

- ▶ Keep a queue of old negatives
- ▶ No need for huge batches
- ▶ Works on 1 GPU!

BYOL (2020)

“Skip the negatives”

- ▶ No negative pairs at all
- ▶ Two networks: student + teacher
- ▶ Simpler, still works

DINO (2021)

“Self-distillation”

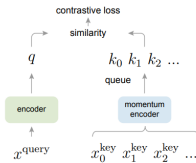
- ▶ Student-teacher like BYOL
- ▶ Multi-scale crops
- ▶ Learns to “see” objects

- ▶ **Problem:** SimCLR needs 4096+ images per batch for enough negatives. That requires many expensive GPUs.
- ▶ **MoCo's trick:** Keep a *memory queue* of recent results
 - Process a small batch (256 images)
 - But compare against 65,536 stored results from previous batches
 - Like keeping a guest list instead of needing everyone in the room at once
- ▶ **Result:** Same quality as SimCLR, but runs on a *single GPU*!

Momentum Contrast for Unsupervised Visual Representation Learning

Kaiming He Haoqi Fan Yuxin Wu Saining Xie Ross Girshick

Facebook AI Research (FAIR)



- ▶ MoCo uses **two copies** of the same network:
 - **Student:** learns quickly (normal training)
 - **Teacher:** updates *very slowly* — moves just 0.1% toward the student each step
- ▶ **Why slow?** The queue stores teacher results. Fast changes → stale, misleading queue entries. Slow teacher keeps everything *consistent*.

The momentum formula

$$\theta_{\text{teacher}} \leftarrow 0.999 \cdot \theta_{\text{teacher}} + 0.001 \cdot \theta_{\text{student}} \quad (99.9\% \text{ old} + 0.1\% \text{ new})$$

- ▶ **BYOL** (2020) asks a bold question:
Do we even need negative pairs?
- ▶ **The setup:** Two networks, two crops of the same image
 - **Student:** sees crop A, tries to predict what the teacher sees
 - **Teacher:** sees crop B, provides the target
 - Teacher updates slowly (momentum trick)
- ▶ **The loss:** make the student's prediction match the teacher's output. No negatives!

Bootstrap Your Own Latent A New Approach to Self-Supervised Learning

Jean-Bastien Grill^{1,2} Florian Strub^{1,2} Florent Alché^{1,2} Corentin Tallec^{1,2} Pierre H. Richemond^{1,2}

Elena Buchatskaya¹ Carl Doersch¹ Bernardo Avila Pires¹ Zhaohan Daniel Guo¹

Mohammad Gheshlaghi Azar¹ Bilal Piot¹ Koray Kavukcuoglu¹ Rémi Munos¹ Michal Valko¹

¹DeepMind

²Imperial College

[jbgrill, fstrub, altche, corentint, richemond]@google.com

Abstract

We introduce Bootstrap Your Own Latent (BYOL), a new approach to self-supervised image representation learning. BYOL relies on two neural networks, referred to as *online* and *target* networks, that interact and learn from each other. From an augmented view of an image, we train the online network to predict the target network representation of the same image under a different augmented view. At the same time, we update the target network with a slow-moving average of the online network. While state-of-the-art methods intrinsically rely on negative pairs, BYOL achieves a new state of the art *without them*. BYOL reaches 74.3% top-1 classification accuracy on ImageNet using the standard linear evaluation protocol with a ResNet-50 architecture and 79.6% with a larger ResNet. We show that BYOL performs on par or better than the current state of the art on both transfer and semi-supervised benchmarks.

- ▶ **The obvious question:** If there are no negatives, why not output the *same thing for every image*? That would make the student always match the teacher!
- ▶ **Three things prevent this “collapse”:**
 1. **Extra prediction step:** The student must *predict* the teacher's output through a small extra network — forces it to actually understand the image
 2. **Slow-moving teacher:** Gives a stable, consistent signal. If both changed fast, they could drift toward useless outputs
 3. **Batch statistics:** Normalizing outputs across the batch secretly introduces info about other images
- ▶ **Result:** BYOL matches SimCLR and MoCo — without any negatives!

- ▶ **DINO** (2021) uses the student-teacher idea from BYOL, but with a clever twist:
- ▶ **The teacher sees the big picture, the student sees details:**
 - Teacher gets large crops (most of the image)
 - Student gets small crops (zoomed-in pieces)
 - Student must figure out: “what is the whole image about, given just this small piece?”
- ▶ **Think of it like this:** Imagine seeing just a cat’s ear and having to describe the whole photo. You would need to *really understand cats*.
- ▶ Teacher updates slowly (same momentum trick as MoCo and BYOL)

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron^{1,2} Hugo Touvron^{1,3} Ishan Misra¹ Hervé Jegou¹
Julien Mairal² Piotr Bojanowski¹ Armand Joulin¹
¹ Facebook AI Research ² Inria* ³ Sorbonne University

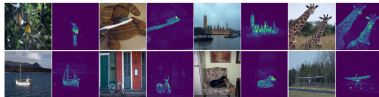


Figure 1: Self-attention from a Vision Transformer with 8×8 patches trained with no supervision. We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

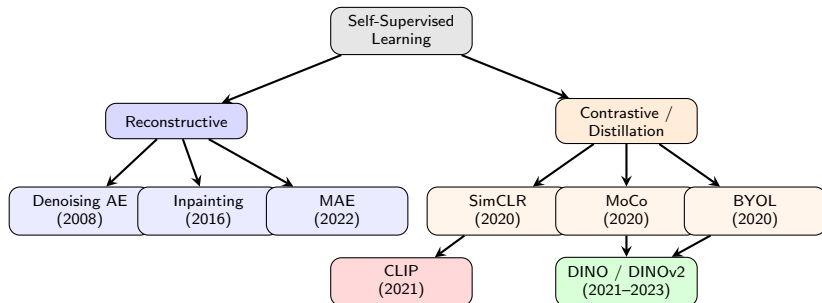
- ▶ **The remarkable discovery:**
 - DINO with ViT learns to *segment objects* automatically
 - Nobody told it where objects are
 - The attention maps naturally focus on the foreground object
- ▶ **Why this matters:**
 - Traditional segmentation needs pixel-level labels (very expensive)
 - DINO discovers object boundaries with *zero labels*
 - This suggests SSL learns something deep about how images are structured
- ▶ **DINOv2** (2023) scaled this up to 1B+ images, producing some of the best general-purpose visual features available

Table 2: **Linear and k -NN classification on ImageNet.** We report top-1 accuracy for linear and k -NN evaluations on the validation set of ImageNet for different self-supervised methods. We focus on ResNet-50 and ViT-small architectures, but also report the best results obtained across architectures. * are run by us. We run the k -NN evaluation for models with official released weights. The throughput (im/s) is calculated on a NVIDIA V100 GPU with 128 samples per forward. Parameters (M) are of the feature extractor.

Method	Arch.	Param.	im/s	Linear	k -NN
Supervised	RN50	23	1237	79.3	79.3
SCLR [11]	RN50	23	1237	69.1	60.7
MoCov2 [13]	RN50	23	1237	71.1	61.9
InfoMin [54]	RN50	23	1237	73.0	65.3
BarlowT [66]	RN50	23	1237	73.2	66.0
OBoW [21]	RN50	23	1237	73.8	61.9
BYOL [23]	RN50	23	1237	74.4	64.8
DCv2 [9]	RN50	23	1237	75.2	67.1
SwAV [9]	RN50	23	1237	75.3	65.7
DINO	RN50	23	1237	75.3	67.5
Supervised	ViT-S	21	1007	79.8	79.8
BYOL* [23]	ViT-S	21	1007	71.4	66.6
MoCov2* [13]	ViT-S	21	1007	72.7	64.4
SwAV* [9]	ViT-S	21	1007	73.5	66.3
DINO	ViT-S	21	1007	77.0	74.5

DINO attention maps show the model learns to “see” objects.

- ▶ **Masked Autoencoders** (He et al., 2022) bring BERT's idea to images:
 1. Split the image into patches (like ViT from Day 19)
 2. Randomly mask **75%** of patches — hide most of the image!
 3. Train a ViT to reconstruct the missing patches
- ▶ **Why 75%?** Images have much more redundancy than text
 - 15% masking (like BERT) is too easy — just interpolate from neighbors
 - At 75%, the model must truly *understand* the scene
- ▶ **Results:**
 - ViT-H achieves 87.8% on ImageNet (state-of-the-art)
 - Efficient: only visible 25% of patches go through the encoder
 - Reconstruction-based (not contrastive) — multiple paths to great SSL



- ▶ **Two paths:** Reconstruction (predict pixels) vs. Contrastive (match embeddings)
- ▶ Both converge: SSL works as well as supervised learning
- ▶ **CLIP** (next!) takes contrastive learning cross-modal: images + text

Method	Negatives?	Momentum?	Batch size	ImageNet top-1
SimCLR	✓	✗	4096	76.5%
MoCo v2	✓	✓	256	77.1%
BYOL	✗	✓	4096	78.6%
DINO (ViT)	✗	✓	1024	80.1%
DINOv2 (ViT-g)	✗	✓	–	86.5%

Supervised ResNet-50 baseline: 76.1%

- ▶ **Takeaway:** SSL now *surpasses* supervised learning on ImageNet
- ▶ The field evolved: negatives → no negatives, CNNs → ViTs
- ▶ DINOv2 features are the “foundation” for many vision tasks today
- ▶ All of this with *zero labeled images*!

- ▶ **Self-supervised learning** creates labels from data itself — no humans needed
- ▶ **Pretext tasks** (rotation, jigsaw, inpainting) were clever but limited
- ▶ **Contrastive learning** (SimCLR, MoCo) pulls similar views together, pushes different ones apart
- ▶ **Self-distillation** (BYOL, DINO) works without negatives
- ▶ **The result:** SSL matches or beats supervised pre-training

The SSL Pipeline

1. Pre-train on **unlabeled** images
2. Freeze encoder, add small head
3. Fine-tune on **small labeled** dataset
4. Great performance!

1. In contrastive learning, what is a **positive pair**?
 - A. Two augmented views of the *same* image
 - B. Two images from the same class
 - C. An image and its label
 - D. Two images that look similar
2. Why does SimCLR need **large batch sizes**?
 - A. To speed up training
 - B. To get more *negative examples*
 - C. To fit the model in memory
 - D. To reduce overfitting
3. What is special about BYOL compared to SimCLR?
 - A. It uses bigger models
 - B. It uses more augmentations
 - C. It works *without negative pairs*
 - D. It trains faster

4. In InfoNCE, what does the temperature τ control?
 - A. The learning rate
 - B. How *sharp* the similarity distinctions are
 - C. The batch size
 - D. The number of augmentations
5. What was the most surprising finding about DINO?
 - A. It trains faster than SimCLR
 - B. It uses less memory
 - C. Its attention maps *automatically segment objects*
 - D. It works on text
6. The momentum update $\theta_k \leftarrow 0.999 \cdot \theta_k + 0.001 \cdot \theta_q$ means...
 - A. Trains twice as fast
 - B. Changes *very slowly*, keeping the queue consistent
 - C. Uses a different dataset
 - D. Has fewer parameters

- ▶ Every method we saw learns from *images alone*
- ▶ The model learns: “these two crops are similar” — but has *no idea what anything is called*
- ▶ **The model cannot:**
 - Tell you “this is a cat” (it just knows embeddings)
 - Search images with text or classify unseen categories
- ▶ **What if we connected images to language?**
 - Images have natural language descriptions everywhere on the internet
 - “A photo of a cat sitting on a couch” (alt text, captions, titles)
 - Can we use this as a *free supervisory signal*?

This leads us to CLIP — contrastive learning between images *and* text.

- ▶ **CLIP** (Radford et al., OpenAI, 2021) applies contrastive learning *across modalities*
- ▶ Instead of: image crop vs. image crop (SimCLR)
- ▶ CLIP does: *image* vs. *text description*
- ▶ **Training data:** 400 million image-text pairs from the internet
 - Images with their captions, alt-text, or titles
 - No manual labeling needed — the text is already there!
- ▶ **The result:** A model that understands both images and text in a *shared embedding space*
 - A photo of a cat and “a photo of a cat” → similar embeddings
 - A photo of a cat and “a photo of a car” → different embeddings

Two separate encoders:

1. Image encoder:

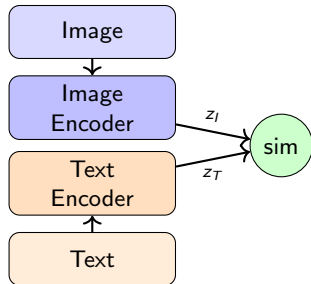
- Takes an image as input
- Outputs a vector (image embedding)
- Can be a ResNet or ViT

2. Text encoder:

- Takes a text caption as input
- Outputs a vector (text embedding)
- A Transformer (like BERT)

Key design:

- ▶ Both embeddings live in the *same vector space*
- ▶ Same dimensionality (e.g., 512)
- ▶ Similarity = cosine similarity



From SimCLR to CLIP: What Changed?

	SimCLR	CLIP
Positive pair	Two crops of same image	Image + its caption
Negative pair	Crops of different images	Image + wrong captions
Encoder	One CNN (shared)	Two encoders (image + text)
Output space	Vision embeddings only	Shared vision-language space
Training data	Unlabeled images	Image-text pairs (web)
How you use it	Add classifier head	Zero-shot with text prompts

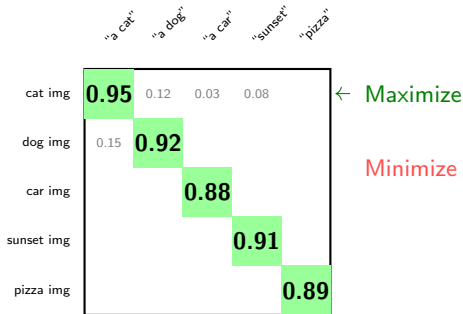
- ▶ **The core loss is identical:** InfoNCE / contrastive
- ▶ **The breakthrough:** By using text as the “other view,” CLIP generalizes to any visual concept describable in words
- ▶ SimCLR learns: “what makes images similar”
- ▶ CLIP learns: “what makes images and descriptions match”

- ▶ **Imagine this game:** You have a batch of images and their captions, all shuffled
 - Each image has exactly *one correct caption*
 - All other captions are *wrong matches*
- ▶ **CLIP plays from both sides:**
 - For each image: “which of these 32,768 captions describes me?”
 - For each caption: “which of these 32,768 images am I about?”
- ▶ **The loss:** Same contrastive loss as SimCLR, but between images and texts

The scale

Each step: 32,768 images vs. 32,768 captions = over **1 billion** possible pairings
— only 32,768 are correct!

- Compute cosine similarity between *every* image and *every* text in the batch:



- *Diagonal*: correct image-text matches → maximize similarity
- *Off-diagonal*: wrong matches → minimize similarity
- Training pushes the diagonal to 1.0 and everything else to 0.0

```
Load pre-trained CLIP model, preprocess = clip.load("ViT-B/32")
Encode an image image = preprocess(Image.open("cat.jpg")).unsqueeze(0)
imageemb = model.encode_image(image)shape : (1, 512)
Encode some text texts = clip.tokenize(["a cat", "a dog", "a car"])
textemb = model.encode_text(texts)shape : (3, 512)
Compare: cosine similarity sims =
(imageembmbtextemb.T)shape : (1, 3)print(sims)Highestfor" acat"!
```

- ▶ **That is it.** Load model, encode image, encode text, dot product.
- ▶ Both embeddings are 512-dimensional vectors in the same space.

- ▶ **Dataset:** 400 million image-text pairs (WIT — WebImageText)
 - Scraped from the internet: images with alt-text captions
 - Not manually curated — noisy but massive
- ▶ **Model:**
 - Image encoder: ViT-L/14 (307M params) + Text encoder: Transformer (63M params)
 - Total: $\sim 370\text{M}$ parameters
- ▶ **Training:**
 - 256 GPUs for 12 days, batch size 32,768
 - Each image contrasted with 32,767 wrong captions
- ▶ **Key point:** The “labels” (captions) were free — already on the web

► Traditional classification:

- Collect labeled training data for your specific classes
- Train a model (or fine-tune a pre-trained one)
- The model can only classify the classes it was trained on

► CLIP zero-shot classification:

- *No training data needed for your task*
- *No fine-tuning*
- Just describe your classes in text!

“**Zero-shot**” means *zero training examples* for the target task.
The model has never seen your specific classes during training.

1. **Create text prompts** for each class: “a photo of a cat”, “a photo of a dog”, “a photo of a car”
2. **Encode** all prompts with the text encoder → text embeddings
3. **Encode** your test image with the image encoder → image embedding
4. **Compute** cosine similarity between the image and each text
5. **Pick** the class with the highest similarity

Example

Image of a tabby cat → encode → compare with:

“a photo of a cat” = **0.92** ✓ “a photo of a dog” = 0.31 “a photo of a car” = 0.05

Prediction: cat! No training needed.


```
model, preprocess = clip.load("ViT-B/32") image =  
    preprocess(Image.open("test.jpg")).unsqueeze(0)  
| Define your classes -- any classes you want! texts = clip.tokenize([ "a photo  
    of a cat", "a photo of a dog", "a photo of a car" ])  
| Encode image and text  
    image_features = model.encode_image(image) text_features = model.encode_text(texts)  
| Cosine similarity -> softmax -> probabilities similarity =  
    (image_features @ text_features.T) probs = similarity.softmax(dim =  
    -1) print(probs) [0.92, 0.05, 0.03]
```

That is it. Change the text prompts → classify anything. No retraining.

- ▶ CLIP zero-shot (no training examples) vs. supervised models (full training):

Dataset	CLIP zero-shot	Supervised baseline
ImageNet	76.2%	76.1% (ResNet-50)
CIFAR-10	95.7%	95.0% (ResNet)
Food-101	92.9%	90.0%
Stanford Cars	77.3%	74.8% (from scratch)
Flowers-102	67.4%	97.0% (fine-tuned)

- ▶ On many benchmarks, CLIP zero-shot *matches* models trained with full supervision
- ▶ On fine-grained tasks (flowers), specialized models still win — but CLIP had *zero* training examples!
- ▶ CLIP is not always the best, but it is the most *flexible*

- ▶ The text prompt matters! Compare:
 - “cat” → okay
 - “a photo of a cat” → better
 - “a photo of a cat, a type of pet” → even better
- ▶ **Why?** CLIP was trained on web captions that usually say “a photo of. . .”
- ▶ **Prompt ensembling:** Use multiple prompts per class and average:
 - “a photo of a cat” + “a close-up photo of a cat” + “a sketch of a cat”
 - Average their embeddings → more robust classification
- ▶ **Same idea as prompt engineering for LLMs** (Day 19) — how you phrase the question affects the answer

- ▶ **Traditional ML workflow** (what you have done all program):
 1. Define your classes
 2. Collect hundreds of labeled examples per class
 3. Split into train/val/test, train for hours
 4. Evaluate — if bad, collect more data
- ▶ **CLIP zero-shot workflow:**
 1. Write your class names as text prompts
 2. Done. Classify.
- ▶ **You can classify things CLIP has never been explicitly trained on**
 - “a satellite image of flooding” — CLIP understands satellite, image, and flooding *separately* and composes them like humans do!

Traditional approach:

1. Collect labeled data
2. Train/fine-tune a model
3. Deploy
4. New class? Go to step 1.

Adding one new class requires retraining!

CLIP approach:

1. Load CLIP (once)
2. Write text prompts for your classes
3. Deploy
4. New class? Just add a text prompt!

Adding a new class takes 5 seconds!

► Real-world impact:

- Content moderation: add new categories on the fly
- Product search: customers describe what they want in words
- Medical: screen for rare conditions without training data
- Creative tools: “find all photos with golden hour lighting”

- ▶ CLIP's shared embedding space enables **cross-modal search**:
 - **Text** → **Image**: “find me a sunset over the ocean” → retrieve matching photos
 - **Image** → **Text**: upload a photo → find the best caption
 - **Image** → **Image**: upload a photo → find similar photos
- ▶ **How it works**:
 1. **Index**: Pre-compute image embeddings for your database
 2. **Query**: Encode the text query with the text encoder
 3. **Search**: Find images closest to the query embedding
 4. **Return**: Top- k most similar images
- ▶ Just *nearest neighbor search* in embedding space — same as Word2Vec (Day 13), but with images and text!

```
2. Search with text query query = clip.tokenize(["a dog playing in snow"])
   query_emb = model.encode_text(query)
3. Find most similar images similarities =
   (query_emb @ image_embeddings.T).topk(5)
4. Return top 5 results for idx in topk.indices : display(all_images[idx])
```

This is exactly what Google Images, Pinterest, and Unsplash do under the hood!

- ▶ **Encoding** (one-time cost per image):
 - CLIP ViT-B/32: ~ 1000 images/second on a GPU
 - 1M images $\rightarrow \sim 17$ minutes to index, only 2 GB of embeddings
- ▶ **Search** (per query):
 - Encode text: ~ 1 ms. Compare with 1M images: ~ 5 ms
 - **Total: under 10ms per query** — faster than you can blink!
- ▶ **For even larger collections** (billions of images):
 - Approximate nearest neighbor libraries (FAISS, ScaNN)
 - Search billions of embeddings in milliseconds
 - This is what Google Images and Pinterest actually do

Before CLIP:

- ▶ Image search used metadata tags
- ▶ Someone had to manually tag every image
- ▶ Search quality limited by tag vocabulary
- ▶ “Sunset over the ocean” only works if someone tagged “sunset” AND “ocean”

▶ Applications everywhere:

- Photo apps (Apple Photos, Google Photos): search your photos with text
- E-commerce: “find me a blue dress with flowers”
- Stock photography and video search

With CLIP:

- ▶ Search directly by content
- ▶ No tagging needed
- ▶ Any natural language query works
- ▶ “A golden retriever wearing a tiny hat” — just works
- ▶ Even novel combinations it has never seen

- ▶ CLIP is not just a classifier or search engine — it is a *foundation*
- ▶ Its shared image-text embedding space enables many downstream applications:

Image Generation

- ▶ **DALL-E / Stable Diffusion:** use CLIP's text encoder to guide image generation
- ▶ Text prompt → CLIP text embedding → diffusion model generates matching image
- ▶ “An astronaut riding a horse on Mars” — CLIP understands the concept, the diffusion model creates it

Visual Question Answering

- ▶ Combine CLIP's image encoder with an LLM
- ▶ Show an image + ask a question
- ▶ The image embedding gives the LLM “eyes”
- ▶ This is how multimodal models (GPT-4V, Gemini, LLaVA) work!

► **Stable Diffusion** (Day 24 preview):

- Text $\xrightarrow{\text{CLIP text encoder}}$ text embedding $\xrightarrow{\text{cross-attention}}$ guide denoising
- CLIP is the “bridge” between your words and the generated image

► **LLaVA, GPT-4V** (multimodal LLMs):

- Image $\xrightarrow{\text{CLIP image encoder}}$ visual tokens $\xrightarrow{\text{concat}}$ feed to LLM
- The LLM can now “see” the image and answer questions about it

► **DALL-E 3, Midjourney:**

- Use CLIP-like encoders to align text descriptions with generated images
- Better text understanding $\rightarrow$ better image generation

► **The pattern:** CLIP taught us how to connect vision and language. Every major multimodal model today builds on this idea.

- ▶ **OpenAI CLIP:** The original. Available via `pip install clip`
- ▶ **OpenCLIP:** Open-source reproduction, many model sizes and datasets
- ▶ **SigLIP:** Google's variant with a simpler loss function
- ▶ **MetaCLIP:** Meta's version trained on curated data

Model	Training data	ImageNet zero-shot
CLIP ViT-B/32	400M pairs	63.2%
CLIP ViT-L/14	400M pairs	75.3%
OpenCLIP ViT-G/14	2B pairs (LAION)	80.1%
SigLIP ViT-SO	4B pairs	83.1%

- ▶ More data + bigger model = better zero-shot performance
- ▶ All available for free via HuggingFace!

- ▶ CLIP is powerful, but it is not perfect:
- ▶ **Counting:** “A photo of 3 cats” vs. “A photo of 5 cats” — CLIP cannot count well
 - It learns *concepts*, not precise quantities
- ▶ **Spatial reasoning:** “A cat on top of a dog” vs. “A dog on top of a cat”
 - CLIP sometimes confuses spatial relationships
- ▶ **Fine-grained distinctions:** Telling apart 200 bird species or 100 car models
 - A specialist model trained on bird images will outperform CLIP
- ▶ **Novel domains:** Medical X-rays, satellite images, microscopy
 - CLIP was trained on *internet* images — it knows less about specialized domains
 - Solution: fine-tune CLIP on domain data (BiomedCLIP, RemoteCLIP, etc.)

CLIP vs. Traditional: When to Use What?

Scenario	CLIP zero-shot	Fine-tuned model
Few or no labeled examples	<i>Best choice</i>	Cannot train
Classes change often	<i>Best choice</i>	Must retrain
Need maximum accuracy	Good baseline	<i>Best choice</i>
Fine-grained classification	Decent	<i>Best choice</i>
Speed matters (real-time)	Fast (one forward pass)	Fast
New domain (medical, satellite)	May struggle	<i>Best with domain data</i>
Prototype / demo	<i>Best choice</i>	Overkill

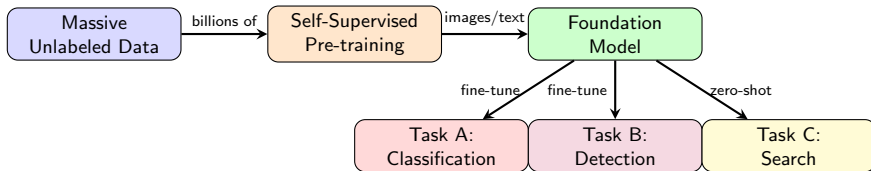
► Best strategy in practice:

1. Start with CLIP zero-shot as your baseline
2. If not good enough, fine-tune CLIP on your labeled data
3. CLIP + LoRA (Day 19!) is the sweet spot for many tasks

- ▶ Almost every AI model you have used this program was pre-trained with SSL:

Model	SSL Method	Day
Word2Vec, GloVe	Predict context words	Day 13
BERT	Masked language modeling	Day 18
GPT	Next-token prediction	Day 18
HuBERT	Masked audio prediction	Day 20
ViT (DINOv2)	Self-distillation on images	Today
CLIP	Image-text contrastive	Today
Stable Diffusion	CLIP + denoising	Day 24

- ▶ **The pattern:** Every modality (text, audio, images, video) benefits from SSL
- ▶ **The trend:** Train on massive unlabeled data, then fine-tune on small labeled sets
- ▶ This is the *foundation model* paradigm — and SSL is what makes it possible



- ▶ **Stage 1:** Pre-train once on huge unlabeled data (expensive, done by big labs)
- ▶ **Stage 2:** Fine-tune cheaply on your specific task (you do this!)
- ▶ Exactly what you did on Day 10 (ResNet), Day 19 (ViT + LoRA), Day 20 (HuBERT + LoRA)

- ▶ You now know many SSL approaches. Here is a practical decision guide:

Your situation	Best approach
Text data (any language)	GPT-style or BERT-style pre-training
Audio data	HuBERT or Whisper + LoRA
Images, need general features	DINOv2 + LoRA (best quality)
Images + text available	CLIP or fine-tune CLIP
Need to classify with no labels	CLIP zero-shot
Need to search images with text	CLIP embeddings + nearest neighbor
Want to generate images from text	CLIP text encoder + Diffusion (Day 24)

- ▶ **Rule of thumb:** Start with the biggest pre-trained model you can find for your modality, then fine-tune with LoRA on your specific task
- ▶ This is the “foundation model” approach — and it works almost everywhere

- After today, you have seen the complete modern AI pipeline:

Technique	What It Does	Day
CNNs	Extract visual features	Day 9
Transfer learning	Reuse pre-trained features	Day 10
Transformers	Model relationships in sequences	Day 17
BERT / GPT	Pre-trained language understanding	Day 18
ViT + LoRA	Efficient vision fine-tuning	Day 19
HuBERT / Whisper	Pre-trained audio understanding	Day 20
SSL (SimCLR, DINO)	Learn features without labels	Today
CLIP	Connect vision and language	Today

- **The power:** Combine these techniques for any problem
- Medical imaging with few labels? DINOv2 + LoRA
 - Content moderation at scale? CLIP zero-shot
 - Audio classification? HuBERT + LoRA (Lab 1 from Day 20!)

Self-Supervised Learning:

- ▶ Labels from data itself
- ▶ Pretext tasks: rotation, jigsaw, inpainting
- ▶ Contrastive: pull similar, push different
- ▶ SimCLR: simple + large batches
- ▶ MoCo: memory queue, small batches
- ▶ BYOL: no negatives needed
- ▶ DINO: self-distillation, emergent segmentation
- ▶ SSL now beats supervised pre-training

CLIP:

- ▶ Contrastive learning across images + text
- ▶ Dual encoder: image encoder + text encoder
- ▶ Shared embedding space
- ▶ Zero-shot classification: just write text prompts
- ▶ Image search with natural language
- ▶ Powers: Stable Diffusion, GPT-4V, LLaVA
- ▶ Foundation for all multimodal AI

1. CLIP's training data consists of:
 - A. Manually labeled images
 - B. *Image-text pairs* from the internet
 - C. Augmented image pairs (like SimCLR)
 - D. Audio-image pairs
2. In CLIP zero-shot, how do you add a new class?
 - A. Retrain the model
 - B. Collect labeled examples
 - C. Just write a *text prompt*
 - D. Add a new output neuron
3. Why does “a photo of a cat” work better than just “cat”?
 - A. It has more words
 - B. CLIP was trained on *web captions* that use this format
 - C. The model prefers longer text
 - D. Coincidence

- ▶ **Cosine similarity** (Day 13, used everywhere today):

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

- ▶ **InfoNCE contrastive loss** (SimCLR, MoCo, CLIP):

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)}$$

- Numerator: similarity with positive. Denominator: all examples. τ : temperature

- ▶ **Momentum update** (MoCo, BYOL, DINO):

$$\theta_{\text{teacher}} \leftarrow m \cdot \theta_{\text{teacher}} + (1-m) \cdot \theta_{\text{student}}$$

► Today's labs:

- Lab 1: Contrastive learning walkthrough — simplified SimCLR, visualize with t-SNE
- Lab 2: Zero-shot classification with CLIP + image search

► Tomorrow (Day 24): GANs + Diffusion Models

- GANs: two networks compete to generate images
- Diffusion: add noise, learn to remove it
- Stable Diffusion uses CLIP's text encoder (today's topic!)

► Day 26: Multimodal Competition

- CLIP is one of your most powerful tools
- Zero-shot features can boost any pipeline

The Three Big Ideas

1. **Self-supervised learning** lets you pre-train on *unlimited unlabeled data*. The model creates its own supervision by solving puzzles.
2. **Contrastive learning** is the most successful SSL approach: similar things close, different things far apart. Features rival supervised learning.
3. **CLIP** extends contrastive learning to connect vision and language — enabling zero-shot classification, image search, and powering modern multimodal AI.

Credits

KAUST Academy

King Abdullah University of Science and Technology